

Flirds 체크포인트 03 — baseline + 선행연구 (PDF 1:1 대조)

원문 PDF/web-extract를 직접 읽고 우리 구현과 1:1 대조. 노트(sources/*.md)만 믿지 않음.

03 · baseline + 선행연구 (PDF 직접 대조)

가장 무거운 문서. 두 부류: (A) 성능비교 경쟁 baseline 7종 · (B) 검출 경쟁 detector 4종 · (C) 정의·구현 참조 선행연구 6편. 각 논문: [목적] · [핵심 원리] · [원문 PDF vs 우리 구현 차이 + 왜] · (후보 중 골랐다면) 왜 이걸. 태그 PDF = 원문 직접 대조 / WEB = web-extract만(PDF 부재) / CODE = 우리 코드 확인.

note→PDF 매핑은 wiki/index.md:151-183에서 확정. Xu(2305.14710)·Bagdasaryan(1807.00459)은 PDF가 디스크에 없고 web-extract만 — 그걸로 대조, 부재 명시.

A. 성능비교 경쟁 valuation baseline (7종)

코드: codes/flirds/baselines/. 전부 같은 열린 궤적 위에서 동작(공정비교). 통합 차이표:

baseline	원논문 PDF	submodel 가중	우리 핵심 변경	왜
GTG-Shapley	2109.02053v1.pdf	within-subset n_k -renorm	MC 구조-guided truncation 충실 이식 ; backend만 LLM(-loss_fn)	충실 포팅
FedSV (Wang2020)	2009.06192v1.pdf	within-subset n_k -renorm	permutation-MC만 (group-testing 생략); normalized variant는 옵션 (기본 off)	GTG가 guided trunc로 포섭; N5/N100엔 group-testing 불필요
Ripple	40034-Article Text-...2026.pdf	FedAvg α_k	sample→client 집계 (Jacobian chain 선형성); eigsh 견고화(ncv/fallback)	client-level 비교 + eigsh flaky
Data Banzhaf	Data Banzhaf_...md (text)	(b)-oracle per-round	MSR estimator 대신 exact $2^{\wedge}N$; (b) coalition util 균등 $1/2^{\wedge}(n-1)$ 재가중	$N \leq 10$ 서 exact 싸고 noise 0; util 고정→kernel만 변수
ShapleyFL	3580305.3599500.pdf	uniform 1/ S	min-max+EMA+uniform-submodel 충실 ; DMC estimator(대N)만 생략	N5 exact 충분; uniform+minmax+EMA가 비-degenerate 만듦
ComFedSV	2109.09046v3.pdf	uniform 1/ S	LIBMF→numpy ALS; from-logs 포팅; partial=True	외부 의존성 제거(동일 목적함수)
loss-heuristic	(논문 없음)	(b) per-round	singleton util $u_{(b)}(\{k\})$	floor baseline; N-R forward 만

교차 통찰 [CODE+⑤]: submodel 가중이 운명을 가른다. within-subset renorm(GTG/FedSV) 은 zero-delta free-rider에도 가중 → $\phi \neq 0$ (희석). per-round FedAvg weight((b)/Banzhaf/loss-heur) 은 zero-delta=0 → ϕ 정확0. N=5 near-additive + (b)-utility 공유 ⇒ GTG/FedSV/Banzhaf/loss-heur 전부 (b)와 **degenerate 동일(+1.000)**;

A.1 GTG-Shapley [PDF 2109.02053v1.pdf §4]

- **목적:** retrain-free federated Shapley(gradient sub-model 재구성 + guided MC)의 정식 — closed-form Taylor 대비 "MC지만 효율적" 경쟁자.
- **원리:** round별 Δ 저장 $\rightarrow$ coalition 모델을 FedAvg 재합으로 재구성(재학습 X). Guided MC = 각 후보를 permutation 0번 자리에 + within-round truncation(marginal $< \epsilon$). 수렴체크로 외부루프 종료.
- **차이/왜:** 알고리즘 deviation 없음(baselines/gtg.py:104-161 permutation guided-truncation:수렴 전부 충실). 유일 확장 = `_round_metrics`서 LLM backend(-loss_fn). **노트 vs PDF:** 노트는 "guided MC"만, PDF §4.2가 first-player 규약 명시 $\rightarrow$ 코드는 PDF 충실.

A.2 FedSV / Principled Federated Data Valuation (Wang 2020) [PDF 2009.06192v1.pdf §4]

- **목적:** federated Shapley의 origin; GTG가 개선하는 per-round permutation-MC + within-round renorm submodel util의 baseline.
- **원리:** round t에서 $s_t(i) = (1/|I_t|) \sum_S C(|I_t|-1, |S|)^{-1} [v(\text{history}+S+i) - v(\text{history}+S)]$, 누적 $s(i) = \sum_t s_t(i)$. permutation 샘플링(Alg.2) 또는 group-testing(Alg.3/4).
- **차이/왜:** permutation-MC만 포팅(group-testing 생략 — N5/N100엔 불필요, GTG가 포섭) [CODE baselines/fedsv.py]. norm-normalized variant는 `normalized=True` 옵션(기본 off, axiom 보존). free-rider $\phi \neq 0$ 은 within-subset renorm의 알려진 귀결(코드는 논문 충실, 비-zero가 논문 그대로의 결과).

A.3 Ripple-Shapley [PDF 40034-Article Text-44125-1-2-20260314.pdf Eqs 5-19, Alg.1]

- **목적:** sample-level single-run federated Shapley(cross-round Jacobian propagation); 가장 비싼 baseline($\sim 42 \times$ Flirds), Spearman 아닌 task-driven(AUROC+runtime)로 평가.
- **원리:** sample z의 drop항(IRDS-style local val-loss 감소) + ripple항(후속 global update Jacobian chain 전파, 저 랭크 subspace Q + eigsh top-k).
- **차이/왜** [CODE baselines/ripple.py(CNN)·ripple_llm.py(LLM)]: ① **sample-level $\rightarrow$ client-level 집계** (Jacobian chain 선형 $\rightarrow$ client = client 내 sample Shapley 합; Flirds가 client-level이라). ② drop/ripple 수식(Eq 5-19) 충실. ③ **eigsh 견고화:** LLM코드는 fixed v0.ncv.ArpackNoConvergence fallback(zero-pad) — CPU spinning stall(알려진 flaky) 대응; CNN코드는 TODO. ④ "62x speedup"은 prior FL-Shapley 대비지 Flirds 대비 아님(재현 안 함). ⑤ LLM은 (rounds,n,P) 전체 materialize $\rightarrow$ N=100엔 stream-projection 필요(연기).

A.4 Data Banzhaf [PDF Data Banzhaf_...md text-extract §4]

- **목적:** Banzhaf 반값(uniform coalition 가중)이 같은 궤적서 Shapley와 다른 ranking을 내는지, noise-robustness 이점이 in-run(deterministic util)서 발현되는지 테스트.
- **원리:** $\phi_{\text{banz}}(i) = (1/2^{n-1}) \sum_{S \subseteq N \setminus i} [U(S \cup i) - U(S)]$. 기여 = safety-margin 정리(noise robustness 최대) + MSR estimator(log 샘플복잡도).
- **차이/왜** [CODE baselines/banzhaf.py]: ① **utility = (a) 재학습 아니라 (b)-oracle coalition util** `_coalition_utilities`(공정비교: 같은 게임). ② **MSR 대신 exact 2^N** ($N \leq 10$ 서 1024 enum 싸고 sampling noise 0; MSR 이점은 util이 비쌀 때만 — 우리는 캐시됨). ③ kernel $1/2^{n-1}$ 충실. ④ zero-delta free-rider ϕ 정확0. **통찰(코드 주석 :9-11):** deterministic util이라 ranking 거의 안 움직임 = 논문 예측 일치, N=5 near-additive로 Shapley $\approx$ Banzhaf 둘 다 +1.000.

A.5 ShapleyFL [PDF 3580305.3599500.pdf Defs 4.1-4.3]

- **목적:** surrogate FSV(uniform submodel + per-round exact Shapley + min-max + EMA). 비-degenerate surrogate가 같은 ranking을 잡는지.
- **원리:** Def4.1 partial FSV(uniform $1/|S|$ submodel, n_k -가중 아님!) $\rightarrow$ Def4.2 per-round min-max $[0,1] \rightarrow$ Def4.3 EMA across rounds(비참여=carry-forward).
- **차이/왜** [CODE baselines/shapleyfl.py]: uniform submodel·min-max·EMA 모두 충실(:40-73). 차이 = ① 공유 exact_shapley 사용(코드중복 회피) ② **from-logs 적용**(공정비교; 논문 in-flight과 수학동일) ③ DMC difference estimator(대N) 생략 $\rightarrow$ cross-device task7. **비-degenerate 이유**(주석 :17-23): uniform util+minmax+EMA가

(b) Shapley와 $\neq$ → Shapley linearity로 안 무너짐 → 실측 +0.86($\neq$ +1.000).

A.6 ComFedSV [PDF 2109.09046v3.pdf §VI, Alg.1]

- 목적: partial participation서 미관측 coalition을 low-rank completion으로 보정 — cross-device용 Flirds의 원리적 대안.
- 원리: utility matrix $U_{T \times 2 \times N} \rightarrow$ low-rank 분해로 결측 채움 $\rightarrow$ completed matrix서 MC permutation Shapley. Assumption1(round0=전원).
- 차이/왜 [CODE `baselines/comfedsv.py`]: LIBMF $\rightarrow$ numpy ALS(동일 목적함수, 외부의존 제거). uniform submodel·loss-decrease sign·permutation 충실. `comfedsv_from_logs`로 우리 log 포맷 적응. `partial=False`(완전관측 exact)는 검증경로. CNN bit-identical, LLM ==exact uniform-Shapley +1.000(R=30).

A.7 loss-heuristic (논문 없음) [CODE `oracle/in_run_sv.py`:54-67, 호출 `phase1_baseline_compare.py`:123]

- 정의: $\phi_{lh}(k) = \sum_r [\ell(w^r + p_k^r \Delta w_k) - \ell(w^r)] =$ (b) 정의 하의 singleton coalition util $U_{\{b\}}(\{k\})$. good $\rightarrow$ low 규약, free-rider ϕ 정확0, $O(N \cdot R)$ forward(coalition 없음). semivalue 아님(LOO 변형). 프로젝트 자작 floor baseline.

B. 검출 경쟁 detector (4종) — threat-matched SUITE

`codes/flirds/baselines/`. 통합표:

detector	원논문 PDF	타깃	model-free?	우리 핵심 변경(PDF에 없는 = 우리 것)
FLDetector	2207.09209v4.pdf (KDD'22)	poisoning	yes	Gap+2-means 생략(AUROC만); ≥ 1 secant pair부터 score; cross-device gap-HVP 적응 (우리 것)
STD-DAGMM	1911.12560v1.pdf (Lin'19)	free-rider(독립)	yes	signed feature-hash 5.6M $\rightarrow$ 256; per-(client,round) pooling; std는 full벡터
FLTrust	2012.13995v3.pdf (NDSS'21)	free-rider+poison	no(val-grad)	signed cosine(ReLU 아님); $g_0 = -\nabla_{val}$ =normalized Flirds-1st
FedDQC	FedDQC_...md (text)	noisy/data-quality	no(client데이터+model)	per-sample filter $\rightarrow$ client-level mean -IRA; hierarchical training 생략

B.1 FLDetector [PDF 2207.09209v4.pdf Alg.1-3]

- 타깃/매칭: model poisoning(crafted-update). poisoning 위협 매칭. (옛 noisy 매칭은 위협 불일치 — `answer_swap`은 정적-나쁜데이터지 crafted 아님; 노트 `fldetector.md`:48 명시).
- 원리: Cauchy-MVT $\hat{g}_i^t = g_i^{t-1} + \hat{\eta}^t(w^t - w^{t-1})$, $\hat{\eta} = L$ -BFGS(Byrd-Nocedal compact, 최근 $N=10$ global 차분). 예측간차 $\|\hat{g} - g\|_2$ per-client $\rightarrow \ell_1$ -norm across clients $\rightarrow$ 최근 N round mean. Gap statistic + 2-means로 malicious cluster 판정.
- 차이/왜 [CODE `baselines/fldetector.py`]: ① model-free server-side from-logs(L-BFGS compact form :51-71 충실, float64 solve). g_i =raw delta, $w^t - w^{t-1} = n_c$ -weighted aggregate, w_r 미사용. ② Gap+2-means 생략(:33-35) — $N=5$ 서 2-means degenerate, 우리는 연속 AUROC. ③ ≥ 1 secant pair부터 score(:99) — $R < 10$ 대응(논문은 50th iter부터). ④ cross-device gap-integrated HVP(우리 것, PDF에 없음 :103-112): client 직전참여 t' 에서 gap $w^r - w^{t'}$ 예측, gap당 HVP 1회 캐시 — full participation서 bit-identical(CNN guard green), cross-device synthetic AUROC=1.0. Flirds와 겹침 없음: FLDetector=시간 일관성(update끼리), Flirds=val-grad 정렬 — 직교 신호.

B.2 STD-DAGMM [PDF 1911.12560v1.pdf §IV-V (Lin 2019)]

- 타깃/매칭: free-rider. gradient 안 쓰는 독립 baseline(FLTrust는 Flirds-1st와 같은 신호라 STD-DAGMM이 독립

측).

- 원리: DAGMM(AE→저차원 z + relative-euclidean + cosine → GMM energy) + **update의 std scalar**를 GMM 입력에 추가. free-rider taxonomy: random weights(Attack I) / delta weights(II) / advanced-delta+노이즈(III).
- 차이/왜 [CODE [baselines/std_dagmm.py](#)]: ① **signed feature-hash 5.6M→256(:61-90)** — LoRA update가 dense AE엔 너무 큼; **std는 full(pre-proj) 벡터서**(magnitude 신호 보존). ② **per-(client,round) pooling(:73-90, N-R sample)** — 논문 per-round은 $N=5$ 서 degenerate(5 sample<2-component GMM); pooling이 partial participation도 자연 처리. ③ z augmentation·GMM energy 충실. ④ unseen client=min score. **우리 위험은 zero/random만**(delta/advanced-delta는 task9). **Flirds와 직교**(순수 model-free, val-grad 안 봄) → 그래서 독립 baseline.

B.3 FLTrust [PDF [2012.13995v3.pdf](#) Alg.2 (Cao 2021)]

- 타깃/매칭: Byzantine(free-rider+poison). cosine-to-root.
- 원리: server가 root dataset으로 g_0 계산 → $\text{trust} = \text{ReLU}(\cosine(g_i, g_0))$ → magnitude normalize → 가중집계. ReLU는 anti-aligned update를 집계서 배제하는 **aggregation gate**.
- 차이/왜 [CODE [baselines/fltrust.py](#)]: ① $g_0 = -\nabla_{\text{val}}(w_r)$ (root→val set; $R_1=1$ 서 수학동일, cosine은 scale-invariant). ② **signed cosine, NOT ReLU(:26-34)** — ReLU는 benign($\cos < 0$)과 free-rider($\cos \approx 0$)를 둘 다 0으로 뭉개 free-rider 검출 깨뜨림; 우리는 AUROC(ranking)라 ReLU/mag-norm은 ranking 불변(집계 gate). ③ **cosine $\approx$ Flirds-1st exact**: signed cosine = $\langle \nabla_{\text{val}}, \Delta w \rangle / (\| \cdot \| \cdot \|) = \text{Flirds-1st의 정규화} = \text{같은 내적의 monotone 변환} \rightarrow \text{ranking 동일} \rightarrow \text{FLTrust는 보조(독립 아님) free-rider baseline. 실측 free-rider AUROC}=1.0(N=100 \text{ 1-seed})$.

B.4 FedDQC [PDF [FedDQC_...md](#) text-extract §4.2]

- 타깃/매칭: noisy/data-quality. IRA가 answer_swap(instruction-response 정렬 깨짐)을 직접 잡음 → 자연 매칭.
- 원리: $\text{IRA} = L(a) - L(a|q)$ (instruction 썼을 때 response loss 감소량). per-sample 품질 필터 + hierarchical training(high→low IRA 순 점진학습).
- 차이/왜 [CODE [baselines/feddqc.py](#)]: ① IRA 정의 **충실(:59 $L(a)-L(a|q)$)**. ② **per-sample filter → client-level mean**, suspicion=-IRA(corrupt=HIGH; :60). 논문은 per-client mean 안 함 — 재용도화(detector). ③ **hierarchical training 전부 생략(:7-8 명시: data-USE는 scope 밖, IRA scorer만)**. ④ subsample 128/client(비용). ⑤ 유일하게 **logs 아니라 client 데이터+model forward 필요** → 가장 비싸고 privacy 약함. noisy AUROC=1.0(1-seed smoke; 단 per-domain IRA 분산 큼 — finance 0.17 $\approx$ noisy 0.067 → matrix서 noisy 도메인/seed 변주).

C. 정의·구현 참조 선행연구 (6편)

C.1 IRDS — In-Run Data Shapley [PDF [Data Shapley in One Training Run.md](#) HTML extract]

- 역할: Flirds의 직접 조상. estimator docstring이 명시([flirds_estimator.py:13](#) "Closed-form approximation of the (b) in-run Shapley").
- 원리: centralized·per-SGD-step·**data-point** level. step t 서 $\phi_z^t = -\eta \nabla \ell(w_t, z^{\text{val}}) \cdot \nabla \ell(w_t, z)$ (1차) + gradient-Hessian-gradient(2차), 1 forward-HVP("ghost" per-sample grad), $\phi_z = \sum_t \phi_z^t$. **true Hessian**. Appx E.2.2: centralized서 2차 거의 무의미(1차 이미 Spearman>0.94).
- 우리 adaptation/차이 **CODE**: centralized→FL, per-step→per-round(multi-step Δw), data-point→client, batch-grad→client aggregate ΔW^r , uniform→FedAvg participant weight $n_k / \sum n$, full→partial participation(round 합), ghost-grad→`torch.func.jvp*grad`(backend-agnostic, eager 필수), any-optimizer→plain SGD mom=0 강제, →LoRA·fp32. 2차 주장 반전: IRDS는 2차 무의미라지만 FL per-round Δw 가 커서 2차 non-trivial(CNN 0.96>0.92로 뒷받침). GGN 테스트 후 기각(IRDS와 일관).
- 왜 IRDS: closed-form Shapley(MC 아님) + single-run(재학습 0) + 1st+2nd true-Hessian을 동시에 만족하는 유일 선행. TraIn=IRDS-1차(non-Shapley), DataInf=iHVP, Ripple=FL서 per-step IRDS지만 eigsh(flaky).

C.2 Koh & Liang 2017 — Influence Functions [PDF [1703.04730v3.pdf](#)]

- **역할:** 배경/대조 — gradient 기반 attribution의 정석, Flirids가 관계되지만 안 쓰는 것.
- **원리:** $I(z, z_{\text{test}}) = -\nabla L(z_{\text{test}})^T H^{-1} \nabla L(z)$. LiSSA로 H^{-1} 근사(명시적 Hessian/역행렬 회피).
- **차이:** IF는 $H^{-1} \cdot v(\text{iHVP})$ — 반복역산, 불안정/비쌈. Flirids는 $H \cdot v(\text{forward HVP}, :111)$ — 1 jvp*grad, 역산 0. IF=고정 0서 점별 marginal; Flirids=궤적 누적 + Shapley 배분. **forward HVP가 iHVP-collapse 원인 회피**의 코드 근거.

C.3 Ghorbani & Zou 2019 — Data Shapley [PDF 1904.02868v2.pdf]

- **역할:** Shapley 프레임 자체 + (a)-retrain oracle 정의 근거.
- **원리:** $\phi_i = C \cdot \sum_S C(n-1, |S|)^{-1} [V(S \cup i) - V(S)]$, $V(S)=S$ 로 **처음부터 재학습**한 모델 성능. null/symmetry/additivity 유일. TMC-Shapley(truncated MC).
- **우리 사용/차이** [CODE `oracle/exact_sv_llm.py`, `phase2_llm_a_oracle.py`]: Shapley 공리 채택(null-player가 free-rider $\phi \approx 0$ 보장), player를 point→client로 lift, $V(S)=\text{coalition FedAvg 재학습 val-loss}$. **핵심차이:** Ghorbani-Zou $V=\text{algorithm-level(학습 randomness 평균)}$; IRDS/Flirids는 model-specific(실제 궤적). → (a)-valloss=(b) + 1.000(같은 게임). **재학습 불가 정량:** (a) $N=10 = \sum C(10, s) \cdot s = 5120$ vs $N=5=80 \rightarrow 2\text{-}5\text{일}/1\text{-GPU} \rightarrow N=100$ 배제 → in-run 동기.

C.4 Xu 2023 — Instructions as Backdoors [WEB 2305.14710-web-extract.md — PDF 부재 확인]

- **역할:** poison 위협의 attack source(instruction-trigger backdoor).
- **원리:** instruction 필드만 poison(clean-label)으로 LLM backdoor. trigger 강도: Induced Instruction > phrase(AddSent) > **rare token(cf/mn/bb/tq/mb)**. ~1% poison, 95.36% ASR, clean 보존. 큰 모델일수록 취약.
- **우리 구현/차이** [CODE `corruptors.py:43-63`]: trigger="tq"(Xu의 **가장 약한** token-level), target=고정 URL string, **poison_frac=clean-preservation knob**(0.5-0.8=clean보존+install, 1.0=clean파괴). 차이: Xu headline=classification+label-match ASR+Induced-Instruction+3epoch+7B; 우리=generative free-form+exact-match ASR+FL+SGD-5step+1B. **노트(instructions-as-backdoors-xu.md)** 명시: "우리 ASR=0은 'Xu 실험 아님'이지 'Xu 반증' 아님". **PDF 디스크에 없음 → web-extract로 대조**(필요시 fetch 가능).

C.5 Bagdasaryan 2020 — How to Backdoor FL [WEB 1807.00459-web-extract.md — PDF 부재 확인]

- **역할:** poison의 FL 전파 메커니즘(model-replacement).
- **원리:** 단일 악성 client가 scaled update로 FedAvg 후 global을 attacker 모델 X로 치환. $\gamma = n/\eta$ (full replacement). stabilizer: 수렴근처 공격 / 낮은 lr / backdoor+benign 혼합 / train-and-scale(norm-bound) / constrain-and-scale(stealthy).
- **우리 구현/차이** [CODE `llm_server.py:57-58, :67`]: $\text{delta} \times \text{attack_scale}(\gamma=\text{cohort size}, N=5\text{서 } \gamma=5)$. **plain-scaled만 — 모든 stabilizer 생략**(norm-bound X, 낮은lr X, 수렴근처 X, constrain-and-scale X). 차이: stealthy arm **불가**(attacker $\parallel \Delta \parallel = 40 \times \text{benign} \rightarrow \text{norm-bound가 backdoor 죽음}$). 목적=Flirids의 clean-val-loss 신호가 작동하는 backdoor를 보는지 norm-defense 회피 시연 아님. LoRA delta만(base 불변). **PDF 부재 → web-extract 대조**.

C.6 Lin et al. 2019 — Free-riders (STD-DAGMM 원논문) [PDF 1911.12560v1.pdf §IV-V]

- **역할:** free-rider 위협의 **attack taxonomy** 근거(detector 측면은 B.2).
- **원리:** Attack I random weights($U[-R, R]$) / II delta weights(연속 global 차분) / III advanced(delta+노이즈, benign std 매칭).
- **우리 구현/차이** [CODE `corruptors.py:70-93`]: **zero**(자명, $\phi=0$ exact) + **random**(scale= $1e-3$, benign-std 매칭은 server call서) = Lin Attack I + trivial. **delta/advanced(II/III) 미구현 → task9**(이전 global aggregate를 FL 루프에 threading 필요). scale은 고정(Lin은 evasion 위해 std-tune). 프레임링 차이: Lin=anomaly 이진검출, 우리 **free_rider=valuation probe**(0/random은 g^r 과 ~직교 → $\phi \approx 0$, signed value의 부산물). MNIST 2-layer MLP만 → **우리가 PEFT-scale 최초 테스트**(노트 명시).

요약: 경쟁 baseline 7종은 전부 같은 열린 궤적 위 충실 포팅(차이는 backend-estimator-vs-exact-util정의로 추적이 능). detector 4종은 threat-matched + 3개 우리-고유 적응(FLDetector gap-HVP / STD-DAGMM hash+pooling / FLTrust signed-cosine). 선행 6편 중 IRDS=직접조상, Koh-Liang/Ghorbani-Zou=대조-oracle근거, Xu/Bagdasaryan/Lin=위협정의(Xu·Bagdasaryan PDF 부재→web-extract).